



Разбор задачи «Непростая энергия кристаллов»

Подзадача 1

Заметим, что в данной подзадаче $n \leq 10$, а значит можно перебрать все возможные распределения кристаллов по камерам. У каждого кристалла есть 2 варианта, в какой камере он может оказаться; выборы делаются независимо, поэтому всего существует 2^n распределений. Воспользуемся стандартным приёмом — перебором масок. Каждый бит маски отвечает за то, в какую камеру попадает тот или иной кристалл: если в i -й позиции маски стоит 0, значит i -й кристалл попадает в первую камеру, иначе — во вторую.

Далее для каждого распределения проверим, что никакая пара кристаллов внутри одной камеры в сумме не даёт простое число. Это можно сделать, перебрав все пары элементов в каждой камере и проверяя их сумму s на простоту (делением на все числа от 2 до $\lfloor \sqrt{s} \rfloor$). Псевдокод:

```
1 for mask = 0 .. (1<n)-1
2   split crystals into two chambers by mask
3   ok = true
4   for each chamber
5     for all pairs (i, j) inside it
6       if isPrime(a[i] + a[j]) then ok = false
7   if ok then output this distribution
8 output -1 -1
```

Итоговая асимптотика:

$$\mathcal{O}(2^n \cdot n^2 \cdot \sqrt{A}),$$

где A — максимальная возможная сумма двух энергий кристаллов.

Подзадачи 2 и 3

В этих подзадачах $n \leq 200$, поэтому наивный перебор распределений уже не подходит, поэтому переведём задачу на язык графов. Представим кристаллы как вершины графа, а ребро будем проводить между вершинами i и j тогда и только тогда, когда $a_i + a_j$ — простое число.

Тогда единственное, что останется сделать, — раскрасить получившийся граф в 2 цвета так, чтобы концы каждого ребра были разноцветными. Если такая раскраска существует, то все вершины одного цвета можно разместить в одну камеру (сумма любых двух энергий внутри одной доли не является простой). Раскраску можно найти с помощью серии обходов в глубину или ширину.

Итоговая асимптотика:

$$\mathcal{O}(n^2 \cdot \sqrt{A}),$$

так как рассматриваются все пары (i, j) и для каждой пары проверяется на простоту $s = a_i + a_j$ как в подзадаче 1, а серия обходов работает за $\mathcal{O}(n^2)$.

В подзадачах 1 и 2 для проверки на простоту сумм s достаточно эффективно было перебирать все числа от 2 до самого s .

Подзадачи 4 и 5

Здесь $n \leq 3000$ и $A \leq 6 \cdot 10^6$, поэтому нужно ускорить проверку простоты. Можно построить решето Эратосфена и предподсчитать для каждого числа от 1 до A , является ли оно простым. Тогда проверка простоты суммы выполняется за $\mathcal{O}(1)$, и остаётся лишь перебрать все пары.

Итоговая асимптотика:

$$\mathcal{O}(A \log \log A + n^2).$$

Такое решение точно подходит для подзадачи 4. Если уменьшить потребление памяти и эффективней реализовать решето, то может подойти и для подзадачи 5.



Подзадача 6 и полное решение

Для полного решения достаточно заметить, что можно все кристаллы с чётной энергией положить в одну камеру, а с нечётной — в другую. Действительно, сумма любых двух чётных чисел чётна и не меньше 4, следовательно, не является простой. Сумма любых двух нечётных чисел также чётна и не меньше 6, значит тоже не является простой. Следовательно, внутри каждой камеры не найдётся пары кристаллов, нарушающей условие.

Итоговая асимптотика:

$$\mathcal{O}(n).$$

Решения, в которых случается переполнение 32-битного типа данных, проходят тесты группы 6.

Разбор задачи «Продажа квартир»

Подзадачи

- $k = 1$;
- $k = t$;
- Все a_i равны;
- Все b_i равны.

При $k = 1$ можем продавать квартиры без ограничений, следовательно продаём, начиная с самых дорогих.

При $k = t$ из каждого района можно продать максимум одну квартиру. Из каждого района выбираем самую дорогую квартиру и только из таких выбираем ответ жадно.

При равных b_i все квартиры из одного района, значит продаём, начиная с самой дорогой, распределяя продажи в дни $1, k + 1, 2k + 1, \dots, xk + 1$.

При равных a_i все квартиры стоят одинаково, а значит нам необходимо продать максимальное количество квартир.

Полное решение

- В каждом районе можно продать не более $\left\lceil \frac{T}{k} \right\rceil$ квартир;
- Причём ровно $\left\lceil \frac{T}{k} \right\rceil + 1$ квартир можно продать только в $T \% k$ районах, а в остальных — максимум $\left\lceil \frac{T}{k} \right\rceil$;
- Перебираем квартиры по убыванию стоимости и жадно включаем квартиру в ответ, если можем (если в соответствующем районе ещё можно продать на 1 квартиру больше).
- Утверждается, что ответ не может быть больше.
- Действительно. Пусть на каждый район наложено одинаковое ограничение $\left\lceil \frac{T}{k} \right\rceil$. Тогда мы получаем задачу о слиянии нескольких отсортированных массивов, которая решается жадно (оставляем в каждом районе только первые $\left\lceil \frac{T}{k} \right\rceil$ квартир). Теперь пусть можем в $T \% k$ районах добрать ещё и $\left\lceil \frac{T}{k} \right\rceil + 1$ квартиру. Тогда заведём ещё один массив, в котором сохраним список всех $\left\lceil \frac{T}{k} \right\rceil + 1$ -х квартир, и ограничим его размер числом $T \% k$. Видно, что оптимальный набор квартир получим жадным алгоритмом. Осталось доказать, что существует расстановка.



- Утверждается, что для полученного набора квартир существует расстановка, не противоречащая правилу.
- Расстановку получим следующим образом. Упорядочим районы по числу квартир по убыванию. Если набранных квартир меньше, чем дней — добавим виртуальные квартиры с нулевой стоимостью, каждая из своего индивидуального района. Теперь будем ставить в самый левый свободный день квартиру из района с наибольшим числом квартир, и в ближайшие $k - 1$ дней не будем учитывать этот район. Тогда в каждый день с 1-го по $T - k + 1$ -го будет доступно для выбора хотя бы k разных районов, при этом максимум из $k - 1$ из них взять квартиру сейчас нельзя.
- Альтернативный способ — сначала распределить квартиры по блокам дней длины k , так чтобы в одном блоке не было двух квартир из одного района. Затем идти по блокам слева направо и расставлять следующим образом: все квартиры из районов, которые встречались в прошлом блоке, ставим на такие же места, остальные ставим произвольно. Заметим, что может быть один меньший по размеру блок — чтобы алгоритм отработал корректно, расстановку необходимо начать с него.

Разбор задачи «Задача Арсения»

Подзадача 1

Переберём все n -значные числа в порядке возрастания. Будем хранить в cnt_S количество рассмотренных n -значных чисел с суммой цифр ровно S . Если после рассмотрения очередного n -значного числа x появилось такое cnt_S , что $cnt_S = k$, то текущий x и есть ответ на задачу. Если перебор закончился, а подобного cnt_S не появилось, то ответа не существует.

Итоговая асимптотика:

$$\mathcal{O}(10^n \cdot n).$$

Подзадачи 2–4

Воспользуемся идеей динамического программирования. Пусть $d_{l,t}$ — это количество строк длины l из цифр с суммой цифр t , причём $0 \leq t \leq 9l$. Тогда:

$$d_{l,t} = \begin{cases} 1, & \text{если } l = 1; \\ \sum_{d=\max(0, t-9(l-1))}^{\min(9, t)} d_{l-1, t-d}, & \text{если } l > 1. \end{cases}$$

Заметим, что если $d_{l,t} \geq k$, то знать точное значение $d_{l,t}$ нет необходимости. Для построения ответа достаточно знать, что $d_{l,t} \geq k$.

Пусть $m(S)$ — это количество n -значных чисел с суммой цифр S , причём $0 \leq S \leq 9n$. Тогда:

$$m(S) = \begin{cases} 1, & \text{если } n = 1; \\ \sum_{D=\max(1, S-9(n-1))}^{\min(9, S)} d_{n-1, S-D}, & \text{если } n > 1. \end{cases}$$

Переберём возможные суммы цифр S . Если $m(S) \geq k$, то найдём x_S — k -е по возрастанию n -значное число с суммой цифр S . Тогда ответом на задачу будет минимальный из найденных x_S .

Построим x_S по цифрам слева направо. Пусть первые pos цифр уже выбраны, а их сумма равна p . На позицию pos пробуем поставить цифру d . После неё должно идти $rem = n - pos - 1$ цифр с суммой цифр $need = S - (p + d)$. Пусть cnt — количество способов дописать эти rem цифр. Тогда $cnt = d_{rem, need}$, если $0 \leq need \leq 9 \cdot rem$. Иначе $cnt = 0$.



Будем перебирать d по возрастанию и поддерживать текущее значение k . Для очередного d смотрим на cnt . Если $cnt < k$, то все числа с этой цифрой идут раньше искомого. Пропустим их целиком, сделав замену k на $k - cnt$, и попробуем следующий d . Если $cnt \geq k$, то искомое k -е число начинается с выбранных pos цифр и цифры d . Фиксируем d и переходим к следующей позиции, увеличив p на d .

Так цифра на каждой позиции однозначно определяется и после n шагов получается число x_S , которое по построению является k -м по возрастанию n -значным числом с суммой цифр S . Псевдокод:

```
1 for pos = 0 .. n-1
2   for d = (1 if pos == 0 and n > 1 else 0) .. 9
3     rem = n - pos - 1
4     need = S - (p + d)
5     if 0 <= need <= 9 * rem
6       cnt = dp[rem][need]
7     else
8       cnt = 0
9
10    if cnt < k
11      k -= cnt
12    else
13      xS[pos] = d
14      p += d
15      break
```

Таким образом, dp считается за $\mathcal{O}(n \cdot 9n) = \mathcal{O}(n^2)$, а перебор всех S и построение k -го числа для каждой подходящей суммы происходит за $\mathcal{O}(9n \cdot n \cdot 10) = \mathcal{O}(n^2)$.

Итоговая асимптотика:

$$\mathcal{O}(n^2).$$

Решения, в которых x_S и ответ на задачу хранятся в 64-битном типе данных, проходят тесты группы 3. А решения, в которых k , x_S и ответ на задачу хранятся в 32-битном типе данных, проходят тесты группы 2.

Подзадачи 5–7 и полное решение

Рассмотрим n -значные числа вида $10^{n-1} + y$, где y — m -значное число, причём $1 < m < n$.

Всего существует $9 \cdot 10^{m-1}$ таких чисел. Сумма цифр числа вида $10^{n-1} + y$ может принимать значения от 2 до $9m + 1$, то есть существует $9m$ различных значений этой суммы. Следовательно, найдётся такая сумма цифр, которая у чисел данного вида встречается хотя бы

$$\left\lceil \frac{9 \cdot 10^{m-1}}{9m} \right\rceil = \left\lceil \frac{10^{m-1}}{m} \right\rceil$$

раз. Поэтому, если выполнено

$$10^{m-1} \geq m \cdot k,$$

то ответ на задачу имеет вид $10^{n-1} + y$. Остаётся только найти y .

Так как $k \leq 10^{18}$, то для полного решения достаточно рассмотреть m до $\min(21, n)$, взять минимальный m , при котором ответ существует (если такого нет, то ответа не существует), после чего найти y можно с помощью решения подзадачи 4. Остаётся вывести $10^{n-1} + y$, если $m < n$, а иначе — y . Если $m < n$, то десятичная запись такого числа начинается с единицы, потом идёт $n - m - 1$ нулей, а затем идут m цифр числа y .

Также надо не забыть рассмотреть n -значные числа вида $10^{n-1} + y$, где y — это цифра, то есть $m = 1$. Число такого вида является ответом только при $k = 1$. В этом случае $y = 0$, то есть ответ равен 10^{n-1} .

Так как m — константа, y находится за $\mathcal{O}(m^2) = \mathcal{O}(1)$, а ответ выводится за $\mathcal{O}(n)$.

Итоговая асимптотика:

$$\mathcal{O}(n).$$



Решения, в которых y хранится в 64-битном типе данных, проходят тесты группы 7. Решения, в которых k и y хранятся в 32-битном типе данных, проходят тесты группы 6. Решения, в которых для нахождения y используется перебор, а не динамическое программирование, проходят тесты группы 5.

Разбор задачи «Слон Филимон и цифра три»

Краткое условие

Дано число n . За одну операцию можно прибавить к нему число вида 3, 33, 333, 3333, ... то есть число, состоящее только из цифр 3. Число называется красивым, если все его цифры не превосходят 3, то есть принадлежат множеству $\{0, 1, 2, 3\}$. Требуется найти минимальное число операций, после которых число станет красивым.

Разбор подзадач

Подзадача 1

При маленьких ограничениях можно просто моделировать сложение “в столбик” и делать динамическое программирование по разрядам.

Пусть $dp[i][add][carry]$ — можно ли обработать i младших разрядов, если:

- add — сколько операций ещё влияет на текущий разряд,
- $carry$ — перенос из предыдущего разряда.

Если текущая цифра равна s_i , то после прибавлений получаем:

$$s_i + 3 \cdot add + carry.$$

Нужно, чтобы последняя цифра была не больше 3:

$$(s_i + 3 \cdot add + carry) \bmod 10 \leq 3.$$

Новый перенос:

$$\left\lfloor \frac{s_i + 3 \cdot add + carry}{10} \right\rfloor.$$

После этого можно выбрать, сколько операций продолжатся дальше.

Сложность такого решения порядка

$$O(\text{len}^4(n)).$$

Подзадачи 2–4

Можно оптимизировать динамику, если хранить не булево значение, а минимальное число операций:

$$dp[i][carry] = \text{минимальное число операций}.$$

Это уменьшает размерность и даёт решение порядка $O(\text{len}^3(n))$.

Подзадача 5 (ответ ≤ 2)

Если известно, что ответ не превосходит 2, можно просто проверить:

- можно ли за 0 операций,
- за 1 операцию,
- за 2 операции.

Это делается перебором чисел вида 3, 33, ...



Подзадачи 6–8

Можно зафиксировать число операций a и проверить, можно ли сделать число красивым.
Это уже приводит к основной идее полного решения.

Полное решение

Шаг 1. Умножим всё на 3

Если число красиво (его цифры ≤ 3), то после умножения на 3 его цифры будут лежать в множестве $\{0, 3, 6, 9\}$.

Одна операция прибавления числа вида $33\dots 3$ после умножения на 3 превращается в прибавление числа $99\dots 9 = 10^k - 1$.

Значит, задача эквивалентна следующей:

Из числа $3n$ нужно за минимальное число операций получить число, состоящее только из цифр $\{0, 3, 6, 9\}$, если за одну операцию можно прибавить число вида $10^k - 1$.

Шаг 2. Разложим операцию

Если сделано a операций, то суммарно мы прибавили

$$(10^{k_1} - 1) + \dots + (10^{k_a} - 1) = (10^{k_1} + \dots + 10^{k_a}) - a.$$

Значит, после a операций получаем число $3n - a + \sum 10^{k_i}$.

То есть задача сводится к следующей:

Рассмотрим число $m = 3n - a$. Можно ли за a прибавлений степеней десяти получить число, состоящее только из цифр $\{0, 3, 6, 9\}$?

Шаг 3. Что делает прибавление 10^k

Прибавление 10^k увеличивает одну цифру на 1, а переносы превращают некоторые 9 в 0. Это допустимо, потому что и 0, и 9 разрешены.

Значит, можно считать, что одна операция увеличивает некоторую цифру на 1.

Шаг 4. Сколько нужно увеличить цифру

Пусть цифра равна d . Тогда минимальное число увеличений, чтобы она стала одной из $\{0, 3, 6, 9\}$:

$$\text{need}(d) = (3 - d \bmod 3) \bmod 3.$$

Определим:

$$\text{dist}(x) = \sum \text{need}(\text{цифры } x).$$

Лемма

Число операций a достаточно тогда и только тогда, когда

$$\text{dist}(3n - a) \leq a.$$

Доказательство

Каждое прибавление степени десяти увеличивает одну цифру на 1, значит, чтобы исправить все цифры, нужно хотя бы dist операций.

Если $\text{dist} \leq a$, можно сначала исправить все цифры, а оставшиеся операции применять к старшим разрядам — переносы не испортят допустимые цифры.

Следовательно, условие является необходимым и достаточным.



Алгоритм

1. Посчитать число $3n$.
2. Посчитать $dist(3n)$.
3. Для $a = 0, 1, 2, \dots$:
 - если $dist(3n - a) \leq a$, вывести a ;
 - иначе уменьшить число на 1 и обновить $dist$.

Разбор задачи «Шустрик и коробки»

Заметим, что для подсчёта суммы произведений по четыре достаточно уметь объединять информацию о двух компонентах связности при их объединении. Действительно, если A_1 — сумма чисел в первой компоненте, A_2 — сумма произведений по два числа в первой компоненте, A_3 — сумма произведений по три числа, A_4 — сумма произведений по четыре числа, а B_1, B_2, B_3, B_4 — суммы произведений во второй компоненте, тогда в компоненте C , полученной объединением компонент A и B , можно подсчитать суммы произведений следующим образом:

$$C_1 = A_1 + B_1$$

$$C_2 = A_2 + B_2 + A_1 \cdot B_1$$

$$C_3 = A_3 + B_3 + A_1 \cdot B_2 + A_2 \cdot B_1$$

$$C_4 = A_4 + B_4 + A_1 \cdot B_3 + A_2 \cdot B_2 + A_3 \cdot B_1$$

Будем перебирать коробки и для каждого набора рёбер подсчитывать нужное значение.

Для ускорения решения достаточно поддерживать DSU с откатами (СНМ, система непересекающихся множеств) и воспользоваться идеей переливания (рёбер) из самой большой подкоробки. Для этого достаточно запустить DFS по вложенным коробкам, в первую очередь заходить в самые маленькие подкоробки и затем откатывать состояние DSU к состоянию до обработки этой подкоробки, после чего подсчитать ответ в самой большой подкоробке и оставить все вложенные в неё рёбра. После этого добавить все рёбра из остальных подкоробок в DSU . Получим решение за $n \log^2 n$.

Существует решение за $n \log n \cdot \alpha(n)$, где $\alpha()$ — асимптотика работы DSU с двумя эвристиками (т.е. без использования откатов вообще). Как реализовать такое решение остаётся упражнением для читателя.